Monitoring Privacy-Preserving Constraints in Microservices Architecture Through Parameter Formalisation

Elena Troubitsyna
Dept. of Computer Science
KTH – Royal Institute of Technology, Sweden
Stockholm, Sweden
elenatro@kth.se

2023 3rd Intelligent Cybersecurity Conference (ICSC) | 979-8-3503-8217-4/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICSC60084.2023.10349989

***Abstract*—Microservices is an architectural style that promotes structuring an application as a collection of loosely coupled fine-grained services. Due to its support for continuous integration and agile development, it has become increasingly popular in different application domains. Since each microservice typically accesses different data, while composing complex applications, it is hard to monitor which data are getting accessed in the entire application workflow. This raises a serious concern over privacy protection, especially in the domains that require handling sensitive data. To guarantee privacy preservation, we need to identify the constraints that should be monitored by data analysis tools in run-time. In this paper, we demonstrate how formal modelling can be used as a basis for deriving monitoring constraints in the applications developed in the microservices architectural style. We formalise modelling patterns for specifying applications composed of microservices, data privacy constraints and demonstrate how to identify privacy violations in run-time.**

***Index Terms*—privacy protection, privacy constraints monitoring, microservices, formal modelling.**

I. Introduction

The General Data Protection Regulation (GDPR) [8] obliges companies to guarantee that privacy is preserved while handling personal information. While it is an especially important issue in such domains as healthcare, financing etc., which inherently handle a lot of sensitive personal data, it is also relevant in the design of systems that rely, e.g., on personal location services to achieve the desired collaborative behaviour. Hence, a wide range of companies are facing a strong technical challenge to guarantee privacy protection while developing and deploying microservices-based applications.

To achieve the required privacy protection, we need first to identify the privacy constraints and then to ensure that they are satisfied both at the design and run-time. However, it is complex and time-consuming work because often applications are composed of hundreds of microservices. Therefore, a manual ad-hoc identification of parameters and constraints to be monitored for privacy protection is practically unfeasible. In this paper, we propose an approach that supports a fomralisation and consequent automation of monitoring of the privacy-preservation constraints in the microservices-based development.

Microservices [13] is an architectural style that advocates building applications as an orchestration of loosely coupled fine-grained services. The main benefits of the style include a smooth support for modularity, continuous integration and multi-site development. According to the style, an application – an orchestrated complex service – is composed of several microservices. Each microservice accesses some data and hence, as a result, an orchestrated complex service typically accesses a large variety of data. the situation is further complicated but also during the design, verification and debugging of the constituent's microservices. Hence, support is required for continuous monitoring and verification of the preservation of required privacy constraints. Therefore, ensuring privacy protection in complex microservices architectures (MSA) poses a serious technical challenge that cannot be addressed in a manual ad-hoc manner and requires formal systematic support.

In this paper, we propose to develop such support using a formal modelling Event-B [3]. Event-B is a formal state-based modelling framework that has an integrated development environment including a specialised prover and model checker provided by the Rodin platform [20]. The framework allows us to formally define the privacy constraints and verify their preservation. In the static part of the Event-B specification, we define both – the microservices in terms of their data access and the intended overall structure of the microservices architecture. In the dynamic part, we defined privacy preservation constraints and verified whether the scenarios that the analysed application intends to support, preserve privacy constraints. Model checking is used then to generate all feasible scenario execution traces under the constraints defined in the context. Each scenario can either preserve or violate privacy constraints, where a breach of privacy is defined as a reachability problem. As a result, we can explicitly identify the causes of potential privacy breaches, as well as propose and verify corrective solutions. Hence, we provide an input for the monitoring tools that can guarantee privacy preservation in run-time. The proposed approach supports an identification of violations of the desired privacy policy at the architectural level. In its turn, it provides a basis for a subsequent run-time monitoring of privacy preservation in the

979-8-3503-8217-4/23/$31.00 ©2023 IEEE

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:04:43 UTC from IEEE Xplore. Restrictions apply.

complex microservices-based applications.

The rest of the paper is organised as follows. In Section II we briefly introduce the basics of the microservices architectures and present our case study – safe passage of railway level crossing by pedestrians, cars and agriculture machines (Priv-LX). In Section III we formally define the basic concepts of privacy protection in the context of a microservice architecture. In Section IV we give a short overview of the Event-B formalism and its associated model checker ProB. In Section V we demonstrate by Priv-LX example our approach to verifying privacy preservation. In Sections VI and VII, we overview the related work and discuss the proposed approach.

II. Privacy in MSA

MicroService Architecture (MSA) [12] is essentially a fine-grained service-oriented architecture. This new architectural style is quickly growing in popularity. MSA advocates the development of applications as an orchestration of loosely coupled services of small granularity. MSA supports continuous integration and shorter development cycles. Moreover, it enables flexibility in programming language selection and improves service scalability.

The main motivation behind MSA is to overcome difficulties associated with developing and maintaining monolithic architectures because microservices can be developed by different engineers and continuously integrated and deployed into the overall application. Each microservice runs its own process and communicates with the others using such lightweight mechanisms, e.g., REST [8].

Many companies from different application domains have embraced MSA. For instance, in the healthcare domain, it helps to alleviate the problem of maintaining the Electronic Health Records that resulted in unmanageably large and complex systems. Moreover, MSA provides additional flexibility in using externally-developed services for handling large and rich volumes of data. While MSA is undoubtedly beneficial with respect to flexibility and agility, it is inherently problematic with respect to privacy protection which is caused by highly distributed and complex data handling scenarios.

In this paper, we investigate the scenario in which different companies and infrastructure owners require sensitive user information in order to guarantee safety. Safety should be guaranteed even though each user can individually decide on privacy policy, i.e. which data can be disclosed to which party. Any application or company that handles such data should guarantee that these constraints are preserved.

Next we present an example, which illustrates privacy protection issues in the transportation domain. We start by describing the functionality of the system and proceed by discussing its architecture from the MSA point of view.

In rural areas, installing full-fledged equipment for level crossing is very costly and economically not viable. Nevertheless, when a train passes a crossing with a road, the safety of all kinds of users – pedestrians, cars and farming equipment should be ensured. One of the solutions, which is currently discussed is to use geographical location services, positioning services as well as national identification methods to ensure safety.

There are several versions of the envisaged scenario. Let us consider the interactions with the pedestrians. We assume that people living in the neighbouring area can upload a specific application warning about approaching trains. To enable the desired functionality, they should share their living location with the traffic authority and enable tracking of geographical location. The traffic authority can also share the data with the data analytics companies as well as the railway system. The user should give an agreement to different aspects of data sharing and to guarantee safety, it should also enable geo-location data sharing.

The interactions with the cars occur when a car enters the vicinity of a railway crossing. The user of the car (driver) should give permission to disclose the geo-location of the car to the application (running in the cloud) as well as travel speed. To ensure safe and smooth passage of the crossing by both train and the car, the data should be shared with a path planning algorithm which is owned by an external organisation.

Finally, there is a version of a railway crossing with installed barriers that are locked but can be unlocked by the authorised users, such as farmers transporting their agricultural equipment. In this case, a railway and road infrastructure owner should maintain a databased (in the cloud) of the authorised users. The authorised users should authenticate themselves via national id service and obtain a permission to open and close the bars. In this case, the data should be shared with railway operator, cloud provider and nation id verification service.

In all these scenarios, we assume that the disclosure is agreed to be handled by a cloud-based datacenter associated with the given railway crossing. The railway crossing composite service should strictly adhere to the individually-defined privacy policy and provide alternative solutions if safety cannot be ensured under the given policy. Any other additional disclosures are not permitted.

The possible non-authorised disclosures are associated with data handling by different applications. The potential points of disclosure include: statistica data collection service, path planning service as well as the user of geographical position data by third party.

Each organisation in our example handles user's data using a number of different microservices. All manipulations with the data are performed on multiple platforms and using various applications. Though the unauthorized disclosure is not intentional, the given architecture does not have the means to monitor the satisfaction of privacy constraints and prevent it.

In the next section, we present our formal outlook on the problem of verifying the preservation of privacy policies and propose a practical approach enabling the discovery of potential unauthorised disclosures in the microservices scenarios.

III. Formalising Privacy Constraints in MSA

In this section, we propose a formalisation of the key concepts of privacy protection as well as put them in the

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:04:43 UTC from IEEE Xplore. Restrictions apply.



Fig. 1. Consumer-Provider model in MSA

context of MSA.

As described in Section 2, the applications designed in MSA style can be viewed as a collection of independently deployable distributed services. Each service processes and modifies the data along the workflow of different applications.

The services interact using an inter-process communication protocol. The two commonly used protocols are HTTP request/response with resource APIs (when querying most of all), and lightweight asynchronous messaging when communicating updates across multiple microservices. A microservice-based application often uses a combination of these communication styles [15].

The microservices can be integrated in asynchronous messaging scenario such as *Consumer-Provider model* [6]. Consumer and Provider are actually web components exposing APIs or implementing a user interface (UI). Let us discuss a *Consumer-Provider model* in details by considering two microservices, communicating between each other via REST. The first service plays the role of a *Consumer*, while the second plays the role of a *Provider* as depicted in Fig. 1. Microservice ms_1 provides the access to certain data via REST API. Microservice ms_2 correspondingly obtains the data, which it can further process or store and provide to other services via its own API. Therefore, each microservice can be represented as a composition of a data provider and data consumer. An application in the microservice-style is defined as a sequence of *Consumer-Provider* arrangements.

Let MS be a set of all microservices and $DATA_TYPE$ a collection of all data types associated with the given application. Then for each microservice $\mathsf{ms} \in MS$, we can define the set of data types that it can consume and produce as follows:

$$Cons_Data : MS \to \mathbb{P}(DATA_TYPE),$$

$$Prod_Data : MS \to \mathbb{P}(DATA_TYPE).$$

Here $\mathbb{P}$ denotes the powerset constructor.

To build an application, we should ensure compatibility of the composed services, i.e., verify the following condition for each *Consumer-Provider* pair:

$$Cons_Data(ms_2) \subseteq Prod_Data(ms_1).$$

In general, for an application composed of microservices we say that *a scenario s* is *feasible* if for the microservices MS^s, such that $MS^s \subseteq MS$ and a scenario defined as a sequence $seq(MS^s)$, where $\mathsf{size}(seq(MS^s)) = n$, the following holds:

$$\forall\, ms_i.\ i \in 1..n \ \ \wedge ms_i \in MS^s \Rightarrow$$
$$Cons_Data(ms_{i+1}) \subseteq Prod_Data(ms_i)$$

A user defines privacy constraints when she gives her consent to disclose different parts (i.e., elements of $DATA_TYPE$) of her private data to different organisations. Hence, to define data privacy policies, we have to introduce the notion of an organisation. Each organisation has a set of associated microservices via which it provides, obtains and processes certain data. Let ORG be a set of all relevant organisations. Then, for each organisation $org \in ORG$, we can specify a relation

$$DOMAIN : ORG \to \mathbb{P}(MS)$$

defining the subset of all microservices associated with it.

A data privacy policy constrains which data can be accessed by which organisation. Therefore, for each $org \in ORG$, we can define the subset of data, which can be accessed according to the defined policy. To represent this, we introduce the relation O_PERM as follows:

$$O_PERM : ORG \to \mathbb{P}\,(DATA_TYPE).$$

We can map the privacy constraints defined over organisations onto the associated microsevices. We introduce the notion of a *non-curious* organisation if all data accessible via associated microsevices comply with the privacy constraints:

$$\forall\, org, ms.\ org \in ORG \wedge ms \in DOMAIN(org) \Rightarrow$$
$$Cons_Data(ms) \subseteq O_PERM(org) \qquad (1)$$

We also say that an organisation is *non-disclosing* if it does not provide access to the data to a potentially curious (i.e. not non-curious) organisation, i.e.,

$$\forall\, org_1, org_2, ms.\ org_1, org_2 \in ORG, ms \in DOMAIN(org_1) \Rightarrow$$
$$Prod_Data(ms) \subseteq O_PERM(org_2) \qquad (2)$$

The equation (2) says that if an organisation gets access to the data, then it is allowed by a privacy policy. Now we can define privacy preservation for MSA.

Definition. *A microservices architecture is privacy-preserving if conditions (1) and (2) are satisfied.*

It means that any microservice does not try to consume the data that are not permitted for its hosting organisation and does not provide access to the data requested by the microservices from the organisation without the corresponding permission.

For complex microservices architectures, verification of these conditions might be too laborious and time consuming. Hence, to ensure that an application is build according to the desired privacy constraints, we should devise a practical lightweight verification strategy. We propose to rely on the intended scenarios to achieve this goal.

Let s be a feasible scenario which can be verified using the corresponding feasibility condition. Moreover, for every microservice we assume that it accumulates the consumed data within a scenario execution. We can define this accumulated data as a function:

$$Cons_Data_Known : MS \to \mathbb{P}(DATA_TYPE).$$

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:04:43 UTC from IEEE Xplore. Restrictions apply.

Then we say, the scenario s violates privacy if the state, in which condition (3) is satisfied is reachable.

$$\exists\ ms.\ ms \in \mathsf{ran}(\mathsf{seq}(MS^s)) \wedge$$
$$Cons_Data_Known(ms) \not\subseteq O_PERM(DOMAIN^{-1}(ms)) \quad (3)$$

Here ran is the function range operator.

In this section, we have formally introduced the basic concepts required to reason about microservice-style applications and defined conditions, which should be verified to ensure consistency of desired scenarios. We argue that the verification of privacy preservation constraints can be implemented within the Event-B framework with the associated prover and model checker provided by the Rodin platform. In the next section, we give a brief overview of Event-B and in Section V we outline the main steps of the verification procedure in Event-B as well as illustrate an application of the proposed approach by an example.

IV. Background: Event-B and ProB

Event-B is a state-based formal approach that promotes the correct-by-construction development and formal verification by theorem proving [3]. In Event-B, a system model is specified as an *abstract state machine*. An abstract state machine encapsulates the model state, represented as a collection of variables, and defines the events – the operations over the states, i.e., it describes the dynamic system behaviour. The variables are strongly typed by the constraining predicates that, together with other important system properties, are defined as model *invariants*. Usually, a machine has an accompanying component, called a *context*, which includes user-defined sets, constants and their properties given as a list of model axioms.

The dynamic behaviour of the system is defined by a collection of atomic *events*. Generally, an event has the following form:

$$e \ \widehat{=}\ \textbf{any}\ a\ \textbf{where}\ G_e\ \textbf{then}\ R_e\ \textbf{end},$$

where e is the event's name, a is the list of local variables, G_e is the event guard, and R_e is the event action. The guard is a predicate over the local variables of the event and the state variables of the system. The guard defines the conditions under which the event is *enabled*. If several events are enabled at the same time, any of them can be chosen for execution nondeterministically.

Event-B employs a top-down refinement-based approach to system development. A development starts from an abstract specification that nondeterministically models most essential functional requirements. In a sequence of refinement steps, we gradually reduce nondeterminism and introduce detailed design decisions. The consistency of Event-B models, i.e., verification of well-formedness, invariant preservation as well as correctness of refinement steps, is demonstrated by proving the relevant verification theorems – proof obligations [3].

Modelling, refinement and verification in Event-B is supported by an automated tool – Rodin platform [21]. The platform provides the designers with an integrated modelling environment, supporting automatic generation and proving of the proof obligations. Moreover, various Rodin extensions allow the modeller to transform models from one representation to another. They also give access to various verification engines (theorem provers, model checkers, SMT solvers).

For instance, the ProB extension [17] of Rodin supports automated consistency checking of Event-B machines via model checking, constraint based checking, and animation. ProB supports an analysis of liveness properties (expressed in linear or computational tree logic (LTL/CTL)), invariant violations as well as the absence of deadlocks. A model checker systematically explores the state space, looking for various errors in the model under consideration [12].

In this paper, we argue that modelling with Event-B provides us with a suitable framework for verification of privacy-preserving policies. In the next section, we formulate the main steps of the verification process and demonstrate its application to the Priv-LX example.

V. Event-B Based Formalisation of Monitoring of Privacy-Preserving Constrains in MSA

In Section III, we have formally defined the main concepts required to verify preservation of privacy constraints in the microservices architecture. In this section, we will map our generic formal definitions onto the Event-B framework and propose the lightweight verification process.

Event-B offers a convenient separation of models of static and dynamic aspects of system behaviour within the unified and hence, semantically coherent specification. This allows us to represent the static and dynamic concepts within the same specification in a consistent way.

In the static part of the specification – context – we aim at defining the mathematical constructs required to represent the microservices architecture under study. An excerpt from the specification pattern of the context is shown in Fig.2. We define the set of all microservices names (ids) as an abstract constant set *MS*. The abstract types of data, to be handled by the desired application, are defined by the abstract set *DATA_TYPE*. Similarly, the names of all the relevant organisations are defined in the abstract set *ORG*.

In the **CONSTANTS** clause, we have an opportunity to explicitly define the static relationships between the abstract sets as the corresponding constants that are defined in as the axioms in the **AXIOMS** clause. The definitions introduced in Section III serve as a basis for achieving this. As we discussed previously, we adopt *Consumer-Producer* pattern for modelling a microservice. Hence, for each microservice we define an abstract relation *Cons_Data*, which maps the microservice name to the set of data types in the abstract set *DATA_TYPE*, which the microservice can consume, i.e., define which data are disclosed to this microservice. Similarly, we define the abstract relation *Prod_Data*, which maps the microsevice name to the set of data types, which the microservice exposes to the other microservices.

The abstract constant *DOMAIN* defines the association between the (name of) organisation and the (names of) microservices. The abstract relation *O_PERM* defines the permissions

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:04:43 UTC from IEEE Xplore. Restrictions apply.

CONTEXT MSA_context
SETS *MS, DATA_TYPE, ORG, ...*
CONSTANTS *Cons_Data, Prod_Data, DOMAIN, O_PERM, ...*
AXIOMS
axm1: $MS \neq \varnothing$
axm2: $DATA_TYPE \neq \varnothing$
axm3: $ORG \neq \varnothing$
axm4: $Cons_Data \in MS \rightarrow \mathbb{P}(DATA_TYPE)$
axm5: $Prod_Data \in MS \rightarrow \mathbb{P}(DATA_TYPE)$
axm6: $DOMAIN \in ORG \rightarrow \mathbb{P}(MS)$
axm7: $O_PERM \in ORG \rightarrow \mathbb{P}(DATA_TYPE)$
axm8: $ScenarioSeq = \{\ldots\}$
...
END

Fig. 2. The excerpt of context MSA_context

for a certain subset of data – a subset of *DATA_TYPE*, to be exposed to each particular organisation.

The introduced definitions formally specify the microservices architectures and privacy constraints. As a result of development or verification, new microservices might be introduced, existing modified or privacy policy changed. These changes can be easily reflected by the corresponding modifications of the context of the specification.

To enable a lightweight verification process, in the context we also statically define the scenario, which should run in our microservices architecture. It is defined as a sequence of (names of) microservices. In principle, we could have represented all the intended scenarios at once. However, it would complicate the verification process, since we would also need to keep track of which scenarios have been verified and failed, which would make the diagnostic of the problems less transparent. Hence, we decided to follow the spirit of continuous integration and propose a workflow, which allows the designer to quickly check one by one the scenarios, which she is interested in.

Now let us describe the dynamic part of the model – the machine. Our goal is to verify that a given scenario that should run in our microservices architectures does not result in the violations of the privacy constraints. In the machine, we define the variables representing the scenarios as a sequence of microservices, progress of a scenario execution as well as the "accumulated" knowledge – the subset of data that is getting exposed to each microservice in the scenario. The types of the introduced variables are defined in the **INVARIANTS** clause. The most interesting part of the invariants is our privacy preservation condition (3), which now can be checked for each scenario that we intend to verify.

The verification process relies on proofs and model checking. Proofs allow us to verify consistency of the introduced constants and correctness of the specification as such. It is a useful mechanism to check correctness of not only the initial specification, but also consequent modifications, which might be caused by the new developments or spotting problems during verification. Model checking is used to iteratively check the desired scenarios. We aim at verifying that none of the scenarios modelled as a sequence of microservices violates the invariant, i.e, condition (3) is preserved by each of them.

The summary of the proposed verification process in Event-B is given below.

A. Approach Description

The proposed approach consists of the following steps:

- *Step 1.* Define (or subsequently modify) the corresponding Event-B *context* component, containing the (types of) involved data elements (in **SETS** and **CONSTANTS** clauses) and the involved static relationships between them (in **AXIOMS** clause). The generic pattern for specifying microservices architecture in the context given in Fig. 2.
- *Step 2.* In the dynamic part, create an abstract Event-B machine that defines the dynamic part of the model. Define the desired privacy preservation constraint as part of the model invariant. Explicitly model the progress of execution of the chosen scenario by defining the events that modify the variables representing the current step and the name of the microservice involved into execution of this step. In the model context define (or modify the definition of) the scenario to be verified as a sequence of microservices. Verify consistency and correctness of the specification by invoking the prover integrated into the Rodin platform.
- *Step 3.* Simulate an execution of a chosen scenario in Event-B machine part by invoking model checker ProB integrated into the Rodin platform. Each scenario can either preserve or violate privacy constraints, where a breach of privacy is defined as a reachability problem. Any violations of the privacy constraints lead to deadlocking the model. If a deadlock is found then diagnose the problem by exploring found counterexample: the execution step of the scenario and the name of the corresponding microservice. Otherwise, go to *Step 5.*
- *Step 4*: Propose a solution to rectify the found problem. It would require to make some changes in the data access rights of some microservices. Feasibility and effectiveness of such changes can be immediately checked using the modified specification. Return to the *Step 2* for necessary modifications of one or several definitions.
- *Step 5*: The verified scenario preserves the defined privacy preservation policy. If all the desired scenarios as-

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:04:43 UTC from IEEE Xplore. Restrictions apply.

sociated with a given application have been verified then the current specification of the data access of the involved microservices preserves the desired privacy constraints and the current version of the application can be safely (from the privacy point of view) deployed. Otherwise, go to *Step 2* to define and verify another scenario.

Below we exemplify the proposed approach using our case study – Priv-LX.

B. Application of the Approach to the Priv-LX Example

To illustrate our approach and, in particular, the application of model checking technique, we take our Priv-LX example discussed in Section II. In this section, we highlight the main parts of the performed development and discuss some of the verification results.

Let us consider the following scenario defined as a sequence of microservices:

$$(\mathsf{Positioning_App}, \mathsf{Path_Planner}, \mathsf{LX_Mng}, \mathsf{Train_Positioning}, \mathsf{LX_Mng}, \mathsf{LX_Statistics}) \quad (4)$$

Here Positioning_App stands for a car positioning service, Path_Planner is a route planner service, LX_Mng is a railway level crossing manager, Train_Positioning service and *LX_Statistics* data collection service of a the railway crossing.

To verify if this scenario satisfies the privacy-preserving constraints (defined as **axm4** and **inv6**), we follow the steps of the approach described in Section V-A. First, we define the static part of our Event-B model – the context component (see Fig. 3). The involved structures (sets of microservices, organisations, data types, etc.) are defined (by **axm1**–**axm3**) by instantiating with the example-specific details the generic specification pattern presented in Fig.3.

Then, we create an abstract Event-B machine model (see Fig. 4). Here we define the artefacts required to model the dynamic aspect, e.g., accumulated data elements *Cons_Data_Known*, a current scenario sequence *SimScenarioCur*, etc. We introduce them as model variables and formulate their properties in the **INVARIANTS** clause.

Next, we define the scenario sequence to be checked as a constant *ScenarioCur* in the model context (see Fig. 3). In the machine part of the Event-B specification of Priv-LX, we simulate this scenario execution by storing a scenario steps in *SimScenarioCur* variable as well as saving the consumed data for every involved in a scenario microservice in *Cons_Data_Known* variable.

To verify a scenario in a traceable manner and facilitate diagnosing the problems, we define a number of events – start, next, compl – that simulate the scenario execution (see Fig. 4). The sequence of microservices is built by starting from the first microservices (modelled by the event start) and simulating the execution sequence leading to the last microservices (modelled by the event next). The scenario execution process is completed when the last (in the current scenario sequence) microservice is executed (modelled by the event compl).

The invariant property **inv7:** $finish = TRUE \Rightarrow SimScenarioCur{=}ScenarioCur$ states that if the scenario execution has been completed then the scenario contains all the steps (i.e., is equal to the executed steps). In the case, when the resulting command sequence *SimScenarioCur* does not match the required sequence, a violation is found by model checking. Consequently, a found scenario sequence becomes an input for *Step 4* for analysis.

For instance, in our example, while checking the railway level crossing by car, we have found a deadlock – the scenario execution deadlocks on the execution of the last miscroservice LX_Statistics. The analysis discovered that the *DATACollector* gets access to the data type $CarDestination$, that is not allowed by the privacy-preserving constraints.

At the current stage, we assume that the changes in the data access rely on the manual inspection of the identified disclosures. However, model animation can significantly facilitate this process by allowing us to replay the identified disclosure scenario and consider the consumed and produces data of each microservice individually.

VI. Related Work

Research on privacy and in particular, privacy preservation has recently attracted significant research attention, see, e.g., [22] for an overview of the state-of-the-art in the area. The authors introduce a three-sphere model of user privacy concerns, relate it to the system operations (data transfer, storage, and processing) and provide the guidelines for building privacy-friendly systems. They identify two main approaches to engineering privacy: *privacy-by-policy* and *privacy-by-architecture*, discuss their technical overlaps and boundaries as well as economic feasibility. Our approach can be classified as former.

Rigorous modelling and reasoning about privacy have also been undertaken in different general as well as domain-specific formal frameworks. Agrafiotis et al. [4] rely on Hoare logic to identify and resolve ambiguities in privacy requirements formulation. In particular, they also consider the problem of inconsistency with the privacy constraints caused by an aggregation of data.

The benefits of rigorous approaches to privacy engineering are discussed, e.g., by [1], [7]. In particular, Abe and Simpson [1] use Z to model system–environment interactions and ProZ to verify whether such interactions can trigger a transition to an undesired privacy-violating state. Privacy is addressed from the "data in the system" perspective, which allows the authors to specify privacy preservation constraints as the model invariant. Our approach adopts a similar view, i.e., we also define privacy as an invariant property of the model.

Our approach combines proof-based verification and model checking. In particular, model checking provides us with a convenient tool for verifying that the scenarios that are intended to be implemented over a given architecture do not reach a privacy violating state, i.e., do not allow unauthorised data disclosure.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:04:43 UTC from IEEE Xplore. Restrictions apply.

CONTEXT PRIV − LX_context
SETS *MS, DATA_TYPE, ORG, S*
CONSTANTS *Cons_Data, Prod_Data, DOMAIN, O_PERM, ScenarioSeq, ScenarioCur...*
AXIOMS
axm1: $MS = \{Positioning_App, Path_Planner, LX_Mng, LX_Statistics, Train_Positioning, RailTraffic_Mng, \ldots\}$
axm2: $DATA_TYPES = \{Car_Position, CarSpeed, CarDestination, TrainPosition, \ldots\}$
axm3: $ORG = \{RailNW, Navig, DataCollector\}$
axm4: $O_PERM = \{RailNW \mapsto \{Train_Position\}, Navig \mapsto \{Car_Position, CarSpeed, CarDestination\}, \ldots\}$
…
axm7: $ScenarioSeq \in S \rightarrow (\mathbb{N} \nrightarrow MS)$
axm8: $ScenarioSeq = \{s_1 \mapsto \{1 \mapsto Positioning_App, 2 \mapsto Path_Planner, 3 \mapsto LX_Mng, 3 \mapsto Train_Positioning, 4 \mapsto LX_Mng, 5 \mapsto LX_Statistics, \}, \ldots\}$
axm9: $ScenarioCur \in \mathbb{N} \nrightarrow MS$
axm10: $ScenarioCur = \{1 \mapsto Positioning_App, 2 \mapsto Path_Planner, 3 \mapsto LX_Mng, 3 \mapsto Train_Positioning, 4 \mapsto LX_Mng, 5 \mapsto LX_Statistics\}$
...

Fig. 3. Case study: context definition

MACHINE PRIV − LX_machine_1
SEES PRIV − LX_context
VARIABLES *Cons_Data_Known, SimScenarioCur, finish, num*
INVARIANTS
…
inv4: $Cons_Data_Known \in MS \rightarrow \mathbb{P}(DATA_TYPE)$ // stores accumulated data elements
inv5: $SimScenarioCur \in \mathbb{N} \nrightarrow MS$ // stores a current scenario sequence
inv6: $\forall ms.\ ms \in ran(ScenarioCur) \Rightarrow Cons_Data_Known(ms) \not\subseteq O_PERM(DOMAIN^{-1}(\{ms\}))$
inv7: $finish=TRUE \Rightarrow SimScenarioCur=ScenarioCur$
…
EVENTS
…
start $\widehat{=}$
any *ms_p, ms_c, org_c, data*
where $num=0 \wedge ms_p = ScenarioCur(1) \wedge ms_c = ScenarioCur(2) \wedge$
$ms_c \in DOMAIN(org_c) \wedge data \subseteq Prod_Data(ms_p) \wedge$
$Cons_Data(ms_c) \subseteq O_PERM(org_c) \wedge \ldots$
then
$SimScenarioCur := \{1 \mapsto ms_p, 2 \mapsto ms_c\}$
$Cons_Data_Known(ms_c) := data$
$num := num + 1$
$finish := bool(num = \mathsf{card}(ScenarioCur))$
end
next $\widehat{=}$
any *ms_p, ms_c, org_c, data*
where $finish = FALSE \wedge num > 0 \wedge$
$ms_p = ScenarioCur(num+2) \wedge ms_c = ScenarioCur(num+2) \wedge$
$ms_c \in DOMAIN(org_c) \wedge data \subseteq Prod_Data(ms_p) \wedge$
$Cons_Data(ms_c) \subseteq O_PERM(org_c) \wedge$
$Cons_Data(ms_c) \subseteq Prod_Data(ms_p) \wedge \ldots$
then
$SimScenarioCur := SimScenarioCur \cup \{num + 1 \mapsto ms_p\}$
$Cons_Data_Known(ms_c) := Cons_Data(ms_c) \cup data$
$num := num + 1$
$finish := bool(num = card(ScenarioCur))$
end
…
END

Fig. 4. Case study: excerpt of the machine Priv_LX_1

Formal modelling of privacy-enhancing protocols in colored Petri Nets is undertaken in [23]. The authors focus on verifying effectiveness of different cryptographic solutions for privacy enhancement. In our work, this problem is addressed by using model checking over the formal specification of the architecture to detect a scenario causing unauthorised disclosure. Metayer et al. [14] present a formal framework for privacy-by-design. They take a multi-agent perspective and define the inference rules for the agents, non trusting each other, to detect fraud. In our work, we have focused on defining more light-weight approach and aimed at supporting automated verification of privacy preservation constraints.

Privacy analysis by verification of Web applications is studied in [9]. This paper presents a logic-based framework for reasoning about the privacy protection provided by a web application. A first order extension of the computation tree

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:04:43 UTC from IEEE Xplore. Restrictions apply.

logic is used to specify a privacy policy. Then a formal transition model is constructed by performing a semi-automatic code level analysis of the web application. The verification relies on Alloy Analyzer and is performed on a finite model, expressed in first order relational logic. In our approach, we aimed at supporting architectural-level analysis and focused on a particular type of architectures – microservices.

A specification of complex privacy policies based on RBAC has been studied in [16]. In this work, elements of a permission (action and resource) for role-based access control (P-RBAC) have been combined with elements of privacy (e.g., purpose, conditions, and obligations). The authors introduce their own formal language to characterise and reason about privacy – requirements based on laws and regulations. This is an interesting theoretical work that can inspire our future work.

A privacy-protecting information system for controlled disclosure of personal data to third parties is proposed in [10]. The authors' contribution mainly consists of two parts: a trustworthy central electronic health-record system and a modified digital water-marking scheme to control and observe data flows after disclosure. Thus, the patients should be able to express and enforce obligations regarding a disclosure of health data to third parties. Secondly, an organization providing electronic health-record system should neither be able to gain access to these health data nor establish a profile about patients. This work relies on tagging the data to enable a consequent forensics analysis. In contrast, in our work, we aim at preventing privacy violations already at the development stage.

In the work [2], an access control model for cooperative healthcare environments is presented. A work-based access control (WBAC) model is proposed by introducing the concept of team role and modifying the user-role assignment model. Verification indicates that WBAC is flexible, easy to manage and secure. In our work, we study the access control and associated privacy policies in the context of miscoservises architecture.

There is a large number of works on formal modelling of RBAC – role-based access control. We overview only a few, which use the same modelling framework – Event-B. Akeel et al. [5] and [11] have studied the problem of data leakage threats in the access control model. They used Event-B and associated refinement approach to formalise the requirements over the specific policy elements that satisfy Confidentiality, Privacy and Trust properties. In our work we use Event-B to formulate and prove the dynamic privacy-protection properties. A data-flow oriented approach to graphical and formal modelling of privacy and security constraints has been proposed in [24]–[26], [29]. These works use the graphical modelling to represent system architecture and the data flow. The diagrams are translated into a formal modelling framework Event-B, to verify the impact of privacy violations and security attacks on the behaviour of the system.

A role-based access control (RBAC) is a popular mechanism for privacy assurance. A contract-based approach to modelling and verification of RBAC for cloud was proposed in [18]. An approach to integrating UML modelling and formal modelling in Event-B to reason about behaviour and properties of web-services was proposed in [19]. A domain-specific language for modelling role-based access control and translating graphical models in Event-B was proposed in [27] and formalised in [28]. In the approach discussed in this paper, we focus on representing a microservices architecture and the privacy policy in connection to the organisational domain, to which a microservice belongs. It enables a more straightforward analysis of the required privacy constraints in the development process.

VII. Conclusions

In this paper, we proposed a formal approach to identifying parameters and constraints to be monitored in run-time to identify privacy violations in the applications developed in the microservices architectural style. We demonstrated how to define the privacy requirements formally and to enable lightweight but rigorous analysis of privacy policy preservation. Our approach supports a clear traceability between the high-level privacy description – the user's consent to disclose particular data to certain organisations – and the formally defined privacy preserving constraints to be monitored in run-time.

Event-B and Rodin platform offered us a suitable framework for the formalisation and automation of the proposed approach. Event-B supports specifying both static and dynamic aspects of the system behaviour within an unified model. This allowed us to specify the data-centric properties of each individual microservice and the overall microservice architecture in the static part of the model. In the dynamic part, we defined privacy preservation constraints and verified whether the scenarios that the analysed application intends to support, preserve privacy constraints. The potential violations are discovered by using model checking, i.e., the causes of the unauthorised data disclosure can be explicitly identified. It helps to rectify the privacy-related problems at the early development stages. Typically, this would require some changes in the data access rights of some microservices. Feasibility and effectiveness of such changes can be immediately checked using the modified specification.

The automated tool support – the Rodin platform – have been critical for implementing our approach. The provers have been used to verify consistency of the architectures and Pro-B model checker to verify scenarios. We believe that integration of these techniques allowed us to create a useful approach to a formal analysis of privacy preservation constraints by the microservices architectures. As a future work, we are planning to investigate privacy preservation of big data analytics methodologies developed in microservice arhcitectural style.

References

[1] Alan Abe and Andrew Simpson. Formal models for privacy. In *Proceedings of EDBT/ICDT 2016*, volume 1558 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2016.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:04:43 UTC from IEEE Xplore. Restrictions apply.

[2] Mohamed Abomhara, Huihui Yang, and Geir M. Køien. Access control model for cooperative healthcare environments: Modeling and verification. In *2016 IEEE International Conference on Healthcare Informatics*, pages 46–54. IEEE Computer Society, 2016.

[3] J.-R. Abrial. *Modeling in Event-B*. Cambridge University Press, 2010.

[4] Ioannis Agrafiotis, Sadie Creese, Michael Goldsmith, and Nick Papanikolaou. Applying formal methods to detect and resolve ambiguities in privacy requirements. In *IFIP Advances in Information and Communication Technology*, volume 352 of *LNCS*, pages 271–282. Springer, 2011.

[5] Fatimah Y. Akeel, Asieh Salehi Fathabadi, Federica Paci, Andrew M. Gravell, and Gary Wills. Formal modelling of data integration systems security policies. *Data Science and Engineering*, 1(3):139–148, 2016.

[6] Antoniya Atanasova. Introduction to Microservices Testing and Consumer Driven Contract Testing with PACT. [Online]. Available at https://www.novatec-gmbh.de/en/blog/introduction-microservices-testing-consumer-driven-contract-testing-pact/. Accessed: 01-April-2019.

[7] Julien Bringer, Hervé Chabanne, Daniel Le Métayer, and Roch Lescuyer. Privacy by design in practice: Reasoning about privacy properties of biometric system architectures. In *FM 2015, Proceedings*, volume 9109 of *LNCS*, pages 90–107. Springer, 2015.

[8] European Commission. Proposal for a General Data Protection Regulation. Codecision legislative procedure for a regulation 2012/0011 (COD), European Commission. Brussels, Belgium (January 2012).

[9] Xiang Fu. Conformance verification of privacy policies. In *Proceedings of the 7th International Workshop on Web Services and Formal Methods (WS-FM 2010)*, volume 6551, pages 86–100. Springer, 2011.

[10] Sebastian Haas, Sven Wohlgemuth, Isao Echizen, Noboru Sonehara, and Günter Müller. Aspects of privacy for electronic health records. *I. J. Medical Informatics*, 80(2):e26–e31, 2011.

[11] Thai Son Hoang, David Basin, and Jean-Raymond Abrial. Specifying access control in event-b. *Technical report*, 624, 2009.

[12] Michael Leuschel and Michael J. Butler. ProB: an Automated Analysis Toolset for the B Method. *STTT*, 10(2):185–203, 2008.

[13] Martin Fowler and James Lewis. Microservices: a definition of this new architectural term. In *[Online]. Available https://martinfowler.com/articles/microservices.ml*. Accessed: 01-April-2019.

[14] Daniel Le Métayer. Privacy by design: a formal framework for the analysis of architectural choices. In *CODASPY'13*, pages 95–104. ACM, 2013.

[15] Microsoft. Communication in a microservice architecture. [Online]. Available at https://docs.microsoft.com/en-us/dotnet/standard/microservices-architecture/architect-microservice-container-applications/communication-in-microservice-architecture. Accessed: 01-April-2019.

[16] Qun Ni, Dan Lin, Elisa Bertino, and Jorge Lobo. Conditional privacy-aware role based access control. In *Computer Security – ESORICS 2007, Proceedings*, volume 4734 of *LNCS*, pages 72–89. Springer, 2007.

[17] ProB. Animator and Model Checker. [Online]. Available at https://www3.hhu.de/stups/prob/index.php/. Accessed: 01-April-2019.

[18] Irum Rauf and Elena Troubitsyna. Generating Cloud Monitors from Models to Secure Clouds. In *DSN 2018, in print*, IEEE Computer Society, 2018.

[19] Irum Rauf, Inna Vistbakka, and Elena Troubitsyna. Formal Verification of Stateful Services with REST APIs using Event-B. In *IEEE ICWS 2018, in print*, IEEE, 2018.

[20] Rigorous Open Development Environment for Complex Systems (RODIN). IST FP6 STREP project: online at http://rodin.cs.ncl.ac.uk/.

[21] Rodin. Event-B platform: online at `http://www.event-b.org/`.

[22] Sarah Spiekermann and Lorrie Faith Cranor. Engineering privacy. *IEEE Trans. Software Eng.*, 35(1):67–82, 2009.

[23] Suriadi Suriadi, Chun Ouyang, Jason Smith, and Ernest Foo. Modeling and verification of privacy enhancing protocols. In *ICFEM 2009. Proceedings*, volume 5885 of *LNCS*, pages 127–146. Springer, 2009.

[24] Anton Tarasyuk, Elena Troubitsyna, and Linas Laibinis. Formal Modelling and Verification of Service-Oriented Systems in Probabilistic Event-B. In *IFM 2012*, pages 237–252. Springer, 2012.

[25] Elena Troubitsyna, Linas Laibinis, Inna Pereverzeva, Tuomas Kuismin, Dubravka Ilic, and Timo Latvala. Towards Security-Explicit Formal Modelling of Safety-Critical Systems. In *SAFECOMP 2016*, volume 9922 of *LNCS*, pages 213–225. Springer, 2016.

[26] Elena Troubitsyna and Inna Vistbakka. Deriving and Formalising Safety and Security Requirements for Control Systems. In *SAFECOMP 2018, in print*, LNCS. Springer, 2018.

[27] Inna Vistbakka, Mikhail Barash, and Elena Troubitsyna. Towards Creating a DSL Facilitating Modelling of Dynamic Access Control in Event-B. In *ABZ 2018*, volume 10817 of *LNCS*, pages 386–391. Springer, 2018.

[28] Inna Vistbakka and Elena Troubitsyna. Modelling and verification of dynamic role-based access control. In Mohamed Faouzi Atig, Saddek Bensalem, Simon Bliudze, and Bruno Monsuez, editors, *Verification and Evaluation of Computer and Communication Systems*, pages 48–63, Cham, 2018. Springer International Publishing.

[29] Inna Vistbakka, Elena Troubitsyna, Tuomas Kuismin, and Timo Latvala. Co-engineering Safety and Security in Industrial Control Systems: A Formal Outlook. In *SERENE 2017*, volume 10479 of *LNCS*, pages 96–114. Springer, 2017.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:04:43 UTC from IEEE Xplore. Restrictions apply.